

Rajalakshmi Engineering College

Name: VISHNU PRIYAN M
Email: 240701604@rajalakshmi.edu.in
Roll no: 240701604
Phone: 6382104971
Branch: REC
Department: CSE - Section 7
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 6_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. What will be the output of the following program?

```
class A {  
    public int i;  
    private int j;  
}  
class B extends A {  
    void display() {  
        super.j = super.i + 1;  
        System.out.println(super.i + " " + super.j);  
    }  
}  
class inheritance {  
    public static void main(String args[]) {  
        B obj = new B();  
        obj.i=1;  
    }  
}
```

```
obj.j=2;  
obj.display();  
}  
}
```

Answer

Compile Time Error

Status : Correct

Marks : 1/1

2. What will be the output of the following Java program?

```
class Test {  
    void show(int a) {  
        System.out.println("Integer method");  
    }  
    void show(String s) {  
        System.out.println("String method");  
    }  
    public static void main(String[] args) {  
        Test obj = new Test();  
        obj.show(null);  
    }  
}
```

Answer

String method

Status : Correct

Marks : 1/1

3. Which of the following is true about method overriding in Java?

Answer

The method must have the same name, same parameters, and must be in different classes with an inheritance relationship

Status : Correct

Marks : 1/1

4. Select the correct keyword for implementing inheritance through the class.

Answer

extends

Status : Correct

Marks : 1/1

5. What will be the output of the following code?

```
class A {  
    int sum(int x) {  
        return x + 2;  
    }  
}
```

```
class B extends A {  
    int sum(int x) {  
        return super.sum(x) * 2;  
    }  
}
```

```
class C extends B {  
    int sum(int x) {  
        return super.sum(x) - 3;  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        C obj = new C();  
        System.out.println(obj.sum(4));  
    }  
}
```

Answer

9

Status : Correct

Marks : 1/1

6. What will be the output of the following Java program?

```
class Vehicle {  
    void startEngine() {  
        System.out.println("Vehicle engine started");  
    }  
}
```

```
class Car extends Vehicle {  
    void startEngine() {  
        System.out.println("Car engine started");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Vehicle myVehicle = new Car();  
        myVehicle.startEngine();  
    }  
}
```

Answer

Car engine started

Status : Correct

Marks : 1/1

7. What will be the output of the following Java program?

```
class A {  
    int value = 10;  
    void display() {  
        System.out.println("A's display: " + value);  
    }  
}
```

```
class B extends A {  
    int value = 20;  
    void display() {  
        System.out.println("B's display: " + value);  
    }  
}
```

```
}  
class Test {  
    public static void main(String[] args) {  
        A obj = new B();  
        obj.display();  
        System.out.println("Value: " + obj.value);  
    }  
}
```

Answer

B's display: 20 Value: 10

Status : Correct

Marks : 1/1

8. What will be the output of the following code?

```
class A {  
    void display() {  
        System.out.println("Display A");  
    }  
}
```

```
class B extends A {  
    void display() {  
        System.out.println("Display B");  
    }  
}
```

```
class C extends B {  
    void display() {  
        super.display();  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.display();  
    }  
}
```

}

Answer

Display B

Status : Correct

Marks : 1/1

9. What will be the output of the following Java program?

```
class Vehicle {
    void start() {
        System.out.println("Vehicle starts");
    }
}
class Car extends Vehicle {

    void start() {
        System.out.println("Car starts");
    }
}
class ElectricCar extends Car {
    void start() {
        System.out.println("Electric Car starts silently");
    }
}
class Test {
    public static void main(String[] args) {
        Vehicle v = new ElectricCar();
        v.start();
    }
}
```

Answer

Electric Car starts silently

Status : Correct

Marks : 1/1

10. Which of the following is the correct way for class B to inherit from class A?

Answer

```
class B extends A {}
```

Status : Correct

Marks : 1/1

11. What will be the output of the following program?

```
class Vehicle {  
    String type = "Vehicle";  
}
```

```
class Car extends Vehicle {  
    String type = "Car";  
}
```

```
class Test {  
    public static void main(String[] args) {  
        Car c = new Car();  
        System.out.println(c.type);  
    }  
}
```

Answer

Car

Status : Correct

Marks : 1/1

12. What will be the output of the following Java program?

```
class Parent {  
    void show() {  
        System.out.println("Parent class");  
    }  
}  
class Child extends Parent {  
    void show() {  
        System.out.println("Child class");  
    }  
}
```

```
}  
class Test {  
    public static void main(String[] args) {  
        Parent obj = new Child();  
        obj.show();  
    }  
}
```

Answer

Child class

Status : Correct

Marks : 1/1

13. What will be the output of the following Java program?

```
class Test {  
    void display(int a, int b) {  
        System.out.println("Method 1");  
    }  
    void display(double a, double b) {  
        System.out.println("Method 2");  
    }  
    public static void main(String[] args) {  
        Test obj = new Test();  
        obj.display(10, 10.0);  
    }  
}
```

Answer

Method 2

Status : Correct

Marks : 1/1

14. What will be the output of the following Java program?

```
class A {  
    void display() {  
        System.out.println("Class A");  
    }  
}
```



```
}  
class B extends A {  
    void show() {  
        System.out.println("Class B");  
    }  
}
```

```
class C extends B {  
    void print() {  
        System.out.println("Class C");  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.display();  
        obj.show();  
        obj.print();  
    }  
}
```

Answer

Class AClass BClass C

Status : Correct

Marks : 1/1

15. What will be the output of the following program?

```
class A {  
    int x = 10;  
}
```

```
class B extends A {  
    int x = 20;  
}
```

```
class C extends B {
```

```
int x = 30;

void display() {
    System.out.println(x);
    System.out.println(super.x);
}

class Test {
    public static void main(String[] args) {
        C obj = new C();
        obj.display();
    }
}
```

Answer

3020

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: VISHNU PRIYAN M
Email: 240701604@rajalakshmi.edu.in
Roll no: 240701604
Phone: 6382104971
Branch: REC
Department: CSE - Section 7
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Elsa subscribes to a premium service with a base monthly cost, a service tax and an extra feature cost. Assist her in writing an inheritance program that takes input for these values and calculates the total monthly cost.

Refer to the below class diagram:

Input Format

The first line of input consists of a double value, representing the base monthly cost.

The second line consists of a double value, representing the service tax.

The third line consists of a double value, representing the extra feature cost.

Output Format

The output prints "Rs. X" where X is a double value, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10.0

2.5

5.0

Output: Rs. 17.50

Answer

```
import java.util.Scanner;

class Subscription {
    protected double baseMonthlyCost;

    public Subscription(double baseMonthlyCost) {
        this.baseMonthlyCost = baseMonthlyCost;
    }
}

class PremiumSubscription extends Subscription {
    private double serviceTax, extraFeatureCost;

    public PremiumSubscription(double baseMonthlyCost, double serviceTax,
double extraFeatureCost) {
        super(baseMonthlyCost);
        this.serviceTax = serviceTax;
        this.extraFeatureCost = extraFeatureCost;
    }

    public double calculateMonthlyCost() {
        return baseMonthlyCost + serviceTax + extraFeatureCost;
    }
}

public class Main {
    public static void main(String[] args) {
```

```
Scanner scanner = new Scanner(System.in);

double baseMonthlyCost = scanner.nextDouble();
double serviceTax = scanner.nextDouble();
double extraFeatureCost = scanner.nextDouble();

PremiumSubscription premiumSubscription = new
PremiumSubscription(baseMonthlyCost, serviceTax, extraFeatureCost);

double totalMonthlyCost = premiumSubscription.calculateMonthlyCost();

System.out.printf("Rs. %.2f%n", totalMonthlyCost);

scanner.close();
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VISHNU PRIYAN M
Email: 240701604@rajalakshmi.edu.in
Roll no: 240701604
Phone: 6382104971
Branch: REC
Department: CSE - Section 7
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Alice is managing an online store and wants to implement a program using inheritance to calculate the selling price of products after applying discounts.

Guide her by following the instructions:

Create a base class called Product with a public double attribute price. Create a subclass called DiscountedProduct, which extends Product and includes a private double attribute discount rate. This subclass has a method called calculateSellingPrice() to determine the final selling price after applying the discount.

Formula: Discounted selling price = price * (1 - discount rate)

Input Format

The first line of input consists of a double value p, the initial price of the product.

The second line consists of a double value d, the discount rate.

Output Format

The output prints "Rs. X", where X is a double value, representing the calculated discounted selling price, rounded off to two decimal places.

If the discount rate is greater than 1, print "Not applicable".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50.00

0.20

Output: Rs. 40.00

Answer

```
import java.util.Scanner;
```

```
class Product {  
    public double price;
```

```
    public Product(double price) {  
        this.price = price;  
    }  
}
```

```
class DiscountedProduct extends Product {  
    private double discountRate;
```

```
    public DiscountedProduct(double price, double discountRate) {  
        super(price);  
        this.discountRate = discountRate;  
    }
```

```
    public double calculateSellingPrice() {  
        if (discountRate > 1) {  
            return -1;  
        }  
    }
```

```
        return price * (1 - discountRate);
    }
}

class ProductPricing {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double initialPrice = scanner.nextDouble();
        double discountRate = scanner.nextDouble();
        DiscountedProduct discountedProduct = new
DiscountedProduct(initialPrice, discountRate);
        double sellingPrice = discountedProduct.calculateSellingPrice();

        if (sellingPrice >= 0) {
            System.out.printf("Rs. %.2f%n", sellingPrice);
        } else {
            System.out.println("Not applicable");
        }
        scanner.close();
    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VISHNU PRIYAN M
Email: 240701604@rajalakshmi.edu.in
Roll no: 240701604
Phone: 6382104971
Branch: REC
Department: CSE - Section 7
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Preethi is working on a project to automate sales tax calculations for items in a store. She wants to create a program that takes the price of an item and the sales tax rate as input and calculates the final price of the item after applying the sales tax.

Write a program using the class SalesTaxCalculator, which contains an overloaded method named calculateFinalPrice to handle both integer and double inputs. The program should also include a Main class that takes user input, calls the appropriate method from SalesTaxCalculator, and prints the final price of the item.

Formula Used: Final price = price + ((price * sales tax rate) / 100)

Input Format

The first line of input consists of an integer price (the price of the item for integer inputs).

The second line of input consists of an integer taxRate (the sales tax rate for integer inputs).

The third line of input consists of a double price (the price of the item for double inputs).

The fourth line of input consists of a double taxRate (the sales tax rate for double inputs).

Output Format

The first line of output prints an integer, representing the final price of the item after applying the sales tax for integer inputs (a and b).

The second line prints a double value, representing the final price of the item after applying the sales tax for double-value inputs (m and n), rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100

10

100.0

5.0

Output: 110

105.00

Answer

```
import java.util.Scanner;
```

```
class SalesTaxCalculator {
```

```
    public static int calculateFinalPrice(int price, int taxRate) {  
        return price + (price * taxRate / 100);  
    }
```

```
    public static double calculateFinalPrice(double price, double taxRate) {  
        return price + (price * taxRate / 100);  
    }
```

```
}  
}  
class Main {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        int intPrice = scanner.nextInt();  
        int intTaxRate = scanner.nextInt();  
        double doublePrice = scanner.nextDouble();  
        double doubleTaxRate = scanner.nextDouble();  
  
        int finalPriceInt = SalesTaxCalculator.calculateFinalPrice(intPrice,  
intTaxRate);  
        double finalPriceDouble =  
SalesTaxCalculator.calculateFinalPrice(doublePrice, doubleTaxRate);  
  
        System.out.println(finalPriceInt);  
        System.out.format("%.2f", finalPriceDouble);  
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VISHNU PRIYAN M
Email: 240701604@rajalakshmi.edu.in
Roll no: 240701604
Phone: 6382104971
Branch: REC
Department: CSE - Section 7
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Mr.Kapoor wants to create a program to calculate the volume of a Cuboid and a Cube using method overriding.

Implements a base class Cuboid with attributes for length, width, and height. Include a method calculateVolume() that computes the volume of the cuboid.

Extends the base class with a subclass Cube representing a cube, where all sides are equal. Override the calculateVolume() method in the Cube class to compute the volume of the cube.

The program should take user input for the dimensions of the cuboid and the side length of the cube and display the calculated volumes with two decimal places.

Input Format

The first line of input consists of 3 space-separated double values, representing the cuboid length, width, and height, respectively.

The second line consists of a double value, representing the side length of the cube.

Output Format

The first line of output prints the volume of the cuboid, rounded off to two decimal places.

The second line prints the volume of the cube, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 60.0 60.0 60.0
50.0

Output: Volume of Cuboid: 216000.00
Volume of Cube: 125000.00

Answer

```
import java.util.Scanner;

class Cuboid {
    double length, width, height;
    Cuboid(double length, double width, double height) {
        this.length = length;
        this.width = width;
        this.height = height;
    }
    double calculateVolume() {
        return length * width * height;
    }
}

class Cube extends Cuboid {
```

```

    Cube(double side) {
        super(side, side, side);
    }
    @Override
    double calculateVolume() {
        return length * length * length;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double cuboidLength = scanner.nextDouble();
        double cuboidWidth = scanner.nextDouble();
        double cuboidHeight = scanner.nextDouble();

        // Regular object instantiation for Cuboid
        Cuboid cuboid = new Cuboid(cuboidLength, cuboidWidth, cuboidHeight);
        System.out.printf("Volume of Cuboid: %.2f\n", cuboid.calculateVolume());

        double cubeSide = scanner.nextDouble();

        // Upcasting - Using superclass reference for subclass object (DMD)
        Cuboid cube = new Cube(cubeSide); // Upcasting
        System.out.printf("Volume of Cube: %.2f", cube.calculateVolume()); // Calls
        Cube's method dynamically
        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VISHNU PRIYAN M
Email: 240701604@rajalakshmi.edu.in
Roll no: 240701604
Phone: 6382104971
Branch: REC
Department: CSE - Section 7
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem statement:

Tim was tasked with developing a grocery shopping app. You have a class hierarchy that includes Item, Produce, and OrganicProduce. Your goal is to calculate the total cost of a shopping list, which may contain a mix of regular produce and organic produce items. Additionally, you need to apply discounts to organic items. Apply a 10% discount on organic produce items

Class Hierarchy:

Item: Base class for all items.

Produce: Subclass of Item for regular produce items.

OrganicProduce: Subclass of Produce for organic produce items.

Input Format

The first line of input consists of an integer, 'n'.

For each 'n' item, the user will provide:

- A string 'type' representing the item type ('Regular' or 'Organic').
- A string 'name' represents the item name.
- A double 'price' represents the item price.

Output Format

The output will display the total cost of the shopping list, including discounts on organic items.

Refer to the sample output for format specifications.

Sample Test Case

Input: 1

Regular Banana 1.99

Output: 1.99

Answer

```
import java.util.Scanner;

class Item {
    String name;
    double price;
    Item(String name, double price) {
        this.name = name;
        this.price = price;
    }
    double calculateCost() {
        return price;
    }
}

class Produce extends Item {
    Produce(String name, double price) {
        super(name, price);
    }
}
```



```

    @Override
    double calculateCost() {
        return price;
    }
}
class OrganicProduce extends Produce {
    OrganicProduce(String name, double price) {
        super(name, price);
    }
    @Override
    double calculateCost() {
        return price * 0.9;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        sc.nextLine(); // Consume newline

        double totalCost = 0.0;

        for (int i = 0; i < n; i++) {
            String type = sc.next();
            String name = sc.next();
            double price = sc.nextDouble();

            if (type.equals("Regular")) {
                Item item = new Produce(name, price);
                totalCost += item.calculateCost();
            } else if (type.equals("Organic")) {
                Item item = new OrganicProduce(name, price);
                totalCost += item.calculateCost();
            }
        }

        System.out.printf("%.2f%n", totalCost);
    }
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VISHNU PRIYAN M
Email: 240701604@rajalakshmi.edu.in
Roll no: 240701604
Phone: 6382104971
Branch: REC
Department: CSE - Section 7
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 6_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

In a company, each manager has a unique employee ID and a monthly salary. You are required to design a program that will calculate and display the annual(12 months) salary of a manager based on the input details provided by the user.

Implement the solution using a single inheritance approach.

Employee: The base class with attributes name and employeeID.

Manager: The derived class inheriting from Employee, with an additional attribute salary.

Input Format

The first line of input consists of a string name, representing the manager's name.

The second line of input consists of an integer employeeID, representing the manager's employee ID.

The third line of input consists of a double salary, representing the manager's monthly salary.

Output Format

The first line of output prints: Name: <name>

The second line of output prints: Annual Salary: Rs. <annual_salary> (rounded to two decimal places).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Davis

234

28750.75

Output: Name: Davis

Annual Salary: Rs. 345009.00

Answer

```
import java.util.Scanner;
import java.text.DecimalFormat;

class Employee {
    String name;
    int employeeID;
    public Employee(String name, int employeeID) {
        this.name = name;
        this.employeeID = employeeID;
    }
}

class Manager extends Employee {
    double salary;
    public Manager(String name, int employeeID, double salary) {
        super(name, employeeID);
        this.salary = salary;
    }
}
```

```

    }
    public double calculateAnnualSalary() {
        return salary * 12;
    }
}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        DecimalFormat df = new DecimalFormat("0.00");

        String name = scanner.nextLine();
        int employeeID = scanner.nextInt();
        double salary = scanner.nextDouble();

        Manager manager = new Manager(name, employeeID, salary);

        System.out.println("Name: " + manager.name);
        System.out.println("Annual Salary: Rs. " +
df.format(manager.calculateAnnualSalary()));

        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

John is planning a long road trip and wants to calculate the distance his car can travel based on its speed and fuel capacity. As John knows that different cars have different fuel efficiencies, he wants a program that can help him estimate the travel distance for any given car.

To do this, you are tasked with creating a program that calculates the travel distance of a car based on its speed and fuel capacity. The calculation is simple and follows the formula:

Travel Distance = Speed * Fuel Capacity

You need to model this system using a Vehicle class and a Car class. The Vehicle class will have attributes for the speed and fuel capacity, while the Car class will inherit from the Vehicle class and include a method to calculate the travel distance.

Input Format

The first line of input consists of a double value representing the speed of the car in km/h.

The second line of input consists of a double value representing the fuel capacity of the car in liters.

Output Format

The first line should print "Speed: X km/h", where X is the speed of the car, rounded to two decimal places.

The second line should print "Fuel Capacity: Y liters", where Y is the fuel capacity of the car, rounded to two decimal places.

The third line should print "Travel Distance: Z km", where Z is the total travel distance the car can cover, rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10.0

1.0

Output: Speed: 10.00 km/h

Fuel Capacity: 1.00 liters

Travel Distance: 10.00 km

Answer

```
import java.util.Scanner;
```

```
class Vehicle {  
    double speed;  
    double fuelCapacity;  
    public Vehicle(double speed, double fuelCapacity) {
```

```

        this.speed = speed;
        this.fuelCapacity = fuelCapacity;
    }
}

class Car extends Vehicle {
    public Car(double speed, double fuelCapacity) {
        super(speed, fuelCapacity);
    }
    public double calculateTravelDistance() {
        return speed * fuelCapacity;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double speed = scanner.nextDouble();
        double fuelCapacity = scanner.nextDouble();

        Car car = new Car(speed, fuelCapacity);

        System.out.println("Speed: " + String.format("%.2f", car.speed) + " km/h");
        System.out.println("Fuel Capacity: " + String.format("%.2f", car.fuelCapacity)
+ " liters");
        System.out.println("Travel Distance: " + String.format("%.2f",
car.calculateTravelDistance()) + " km");

        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Sharon, a software developer, is working on a project to automate velocity calculations for various objects. She wants to create a class named VelocityCalculator with overloaded methods calculateVelocity to calculate the velocity. One method will accept distance in meters and time in seconds as integers, while another will accept distance and time as

doubles.

Help her in completing the project.

Formula: Velocity = distance / time

Input Format

The first line of input consists of an integer, representing the distance in meters (for the integer method).

The second line consists of an integer, representing the time in seconds (for the integer method).

The third line consists of a double value, representing the distance in meters (for the double method).

The fourth line consists of a double value, representing the time in seconds (for the double method).

Output Format

The first line prints the velocity calculated using the integer inputs in the format:

Velocity with integer inputs: <velocity> m/s

The second line prints the velocity calculated using the double inputs in the format:

Velocity with double inputs: <velocity> m/s

Note:

The velocity for the double inputs should be printed with two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100

10

100.5

10.2

Output: Velocity with integer inputs: 10 m/s

Velocity with double inputs: 9.85 m/s

Answer

```
import java.util.Scanner;

class VelocityCalculator {
    public static int calculateVelocity(int distance, int time) {
        return distance / time;
    }
    public static double calculateVelocity(double distance, double time) {
        return distance / time;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int distanceInt = scanner.nextInt();
        int timeInt = scanner.nextInt();

        double distanceDouble = scanner.nextDouble();
        double timeDouble = scanner.nextDouble();

        int velocityInt = VelocityCalculator.calculateVelocity(distanceInt, timeInt);
        double velocityDouble =
        VelocityCalculator.calculateVelocity(distanceDouble, timeDouble);

        System.out.println("Velocity with integer inputs: " + velocityInt + " m/s");
        System.out.printf("Velocity with double inputs: %.2f m/s", velocityDouble);

        scanner.close();
    }
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Ram is designing a program to calculate the Body Mass Index (BMI). Your task is to assist him by following the given specifications.

Create a base class BMIcalculator with a method calculateBMI() to compute BMI using the formula $\text{weight} / (\text{height} * \text{height})$.

Extend the class with a subclass CustomBMIcalculator that overrides the method calculateBMI() to calculate BMI based on custom criteria, assigning categories such as "Underweight," "Normal Weight," "Overweight," or "Obese."

BMI < 18.5, category = "Underweight" BMI >= 18.5 & < 24.9, category = "Normal Weight" BMI >= 25 & < 29.9, category = "Overweight" else category = "Obese"

Implement user input for weight and height and display both the standard and custom BMI calculations.

Input Format

The first line of input consists of a double value, representing the weight in kgs.

The second line consists of a double value, representing the height in meters.

Output Format

The first line of output prints: "Standard BMI Calculation:"

The second line of output prints: "BMI: " followed by the calculated BMI value (to two decimal places).

The third line of output prints: "Custom BMI Calculation:"

The fourth line of output prints: "Category: " followed by the BMI category.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 69.7

2.6

Output: Standard BMI Calculation:

BMI: 10.31

Custom BMI Calculation:

Category: Underweight

Answer

```
import java.util.Scanner;
```

```
class BMIcalculator {  
    double weight;  
    double height;  
    public BMIcalculator(double weight, double height) {  
        this.weight = weight;  
        this.height = height;  
    }  
    public double calculateBMI() {  
        return weight / (height * height);  
    }  
    public void displayBMI() {  
        System.out.println("BMI: " + String.format("%.2f", calculateBMI()));  
    }  
}
```

```
class CustomBMIcalculator extends BMIcalculator {  
    public CustomBMIcalculator(double weight, double height) {  
        super(weight, height);  
    }  
    @Override  
    public double calculateBMI() {  
        return super.calculateBMI();  
    }  
    public void displayCustomBMI() {  
        double bmi = calculateBMI();  
        String category;  
  
        if (bmi < 18.5) {  
            category = "Underweight";  
        }  
        else if (bmi >= 18.5 && bmi < 24.9) {  
            category = "Normal Weight";  
        }  
    }  
}
```

```
        else if (bmi >= 25 && bmi < 29.9) {
            category = "Overweight";
        }
        else {
            category = "Obese";
        }

        System.out.println("Category: " + category);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double weight = scanner.nextDouble();
        double height = scanner.nextDouble();

        BMIcalculator bmiCalculator = new BMIcalculator(weight, height);
        System.out.println("Standard BMI Calculation:");
        bmiCalculator.displayBMI();

        CustomBMIcalculator customBMIcalculator = new
        CustomBMIcalculator(weight, height);
        System.out.println("Custom BMI Calculation:");
        customBMIcalculator.displayCustomBMI();

        scanner.close();
    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VISHNU PRIYAN M
Email: 240701604@rajalakshmi.edu.in
Roll no: 240701604
Phone: 6382104971
Branch: REC
Department: CSE - Section 7
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 6_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

A bank provides two types of deposit schemes: Fixed Deposits (FD) and Recurring Deposits (RD). Customers want to calculate the interest they can earn based on their selected scheme.

Develop a Java program using inheritance to compute the interest for FD and RD. The program should include:

A base class Account with attributes accountHolder and principalAmount, along with a method for interest calculation. A subclass FixedDeposit that calculates interest for FD. A subclass RecurringDeposit that calculates interest for RD.

Formulas Used:

Interest for FD: $(\text{principal amount} * \text{duration in years} * \text{rate of interest}) / 100$

Interest for RD: $(\text{maturity amount} * \text{duration in months} * \text{rate of interest}) / (12 * 100)$, where maturity amount = monthly deposit * duration in months.

Input Format

The first line of input consists of the choice (1 for FD, 2 for RD).

If the choice is 1, the following lines consist of account holder (string), principal amount (double), duration in years (int), and rate of interest (double).

If the choice is 2, the following lines consist of account holder (string), monthly deposit (int), duration in months (int), and rate of interest (double).

Output Format

The output prints the calculated interest with one decimal place in the following format.

For choice 1: "Interest for FD: <calculated interest >"

For choice 2: "Interest for FD: <calculated interest >"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

Alice

50000.56

5

6.5

Output: Interest for FD: 16250.2

Answer

```
import java.util.Scanner;
```

```
class Account {  
    protected String accountHolder;  
    protected double principalAmount;
```

```

public Account(String accountHolder, double principalAmount) {
    this.accountHolder = accountHolder;
    this.principalAmount = principalAmount;
}

public double calculateInterest() {
    return 0.0;
}
}

class FixedDeposit extends Account {
    private int durationYears;
    private double rateOfInterest;

    public FixedDeposit(String accountHolder, double principalAmount, int
durationYears, double rateOfInterest) {
        super(accountHolder, principalAmount);
        this.durationYears = durationYears;
        this.rateOfInterest = rateOfInterest;
    }

    @Override
    public double calculateInterest() {
        return (principalAmount * durationYears * rateOfInterest) / 100;
    }
}

class RecurringDeposit extends Account {
    private int durationMonths;
    private double rateOfInterest;

    public RecurringDeposit(String accountHolder, double monthlyDeposit, int
durationMonths, double rateOfInterest) {
        super(accountHolder, monthlyDeposit);
        this.durationMonths = durationMonths;
        this.rateOfInterest = rateOfInterest;
    }

    @Override
    public double calculateInterest() {
        double maturityAmount = principalAmount * durationMonths;
        return (maturityAmount * durationMonths * rateOfInterest) / (12 * 100);
    }
}

```

```

    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int choice = sc.nextInt();

        switch (choice) {
            case 1:
                sc.nextLine();
                String fdName = sc.nextLine();
                double fdPrincipal = sc.nextDouble();
                int fdDuration = sc.nextInt();
                double fdRate = sc.nextDouble();

                FixedDeposit fd = new FixedDeposit(fdName, fdPrincipal, fdDuration,
fdRate);
                System.out.printf("Interest for FD: %.1f", fd.calculateInterest());
                break;

            case 2:
                sc.nextLine();
                String rdName = sc.nextLine();
                int rdDeposit = sc.nextInt();
                int rdDuration = sc.nextInt();
                double rdRate = sc.nextDouble();

                RecurringDeposit rd = new RecurringDeposit(rdName, rdDeposit,
rdDuration, rdRate);
                System.out.printf("Interest for RD: %.1f", rd.calculateInterest());
                break;

            default:
                System.out.println("Invalid Choice");
        }
    }
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Teena is launching a new airline, Boeing747, and needs to calculate the total revenue generated from ticket sales based on the ticket cost and seat availability. Teena's airline offers two types of seats: regular and premium. The ticket cost and seat availability for both types of seats need to be considered for revenue calculation.

To help with this, Teena wants to implement a system using multilevel inheritance with three classes:

Airline: This class will have the ticket cost as an attribute and defines the method `setCost(double cost)` and `double getCost()`. **Indigo:** This class will extend **Airline** and add the seat availability attribute and defines the method `getSeatAvailability()` and `setSeatAvailability(int seatAvailability)`. **Boeing747:** This class will extend **Indigo** and include a method `calculateTotalRevenue()` based on the ticket cost and seat availability.

Teena needs to calculate the total revenue using the formula:

$$\text{Total Revenue} = \text{ticket cost} * \text{seat availability}$$

Help Teena implement this system for calculating the revenue of her airline.

Input Format

The first line of input consists of a double value, representing the flight's ticket cost.

The second line consists of an integer, representing seat availability.

Output Format

The first line of output prints "Ticket Cost: Rs. " followed by a double value representing the ticket cost rounded to one decimal place.

The second line of output prints "Seat Availability: X seats" where X is an integer value representing the seat availability.

The third line of output prints "Total Revenue: Rs. " followed by a double value representing the total revenue rounded to one decimal place.

Refer to the sample output for the exact text and format.

Sample Test Case

Input: 1000.0

100

Output: Ticket Cost: Rs. 1000.0

Seat Availability: 100 seats

Total Revenue: Rs. 100000.0

Answer

```
import java.util.Scanner;
```

```
class Airline {
```

```
    private double cost;
```

```
    public void setCost(double cost) {
```

```
        this.cost = cost;
```

```
    }
```

```
    public double getCost() {
```

```
        return cost;
```

```
    }
```

```
}
```

```
class Indigo extends Airline {
```

```
    private int seatAvailability;
```

```
    public void setSeatAvailability(int seatAvailability) {
```

```
        this.seatAvailability = seatAvailability;
```

```
    }
```

```
    public int getSeatAvailability() {
```

```
        return seatAvailability;
```

```
    }
```

```
}
```

```
class Boeing747 extends Indigo {
```

```
    public double calculateTotalRevenue() {
```

```
        return getCost() * getSeatAvailability();
```

```
    }
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```

Scanner scanner = new Scanner(System.in);
Boeing747 plane = new Boeing747();

double ticketCost = scanner.nextDouble();
plane.setCost(ticketCost);
int seatAvailability = scanner.nextInt();
plane.setSeatAvailability(seatAvailability);

System.out.printf("Ticket Cost: Rs. %.1f\n", plane.getCost());
System.out.println("Seat Availability: " + plane.getSeatAvailability() + "
seats");
System.out.printf("Total Revenue: Rs. %.1f\n",
plane.calculateTotalRevenue());
}
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Mary is managing a business and wants to analyze its profitability. She operates both a regular business model and a seasonal business model. To assess profitability, she uses a program that calculates and compares the profit margins for both models based on revenue and cost.

The program defines:

BusinessUtility class with a method calculateMargin(double revenue, double cost). SeasonalBusinessUtility (inherits from BusinessUtility) and overrides calculateMargin(double revenue, double cost), adding a seasonal adjustment of 10% to the base margin. ProfitabilityChecker class with a method checkProfitability(double regularMargin), which prints "Business is profitable." if the regular margin is 10% or more, otherwise prints "Business is not profitable.".

Mary inputs revenue and cost, and the program compute and display the regular and seasonal margins using:

Margin = ((Revenue – Cost) / Revenue) × 100

Seasonal Margin = Margin + 10

Input Format

The first line of input consists of a double value r , representing the revenue.

The second line consists of a double value c , representing the cost.

Output Format

The first line prints a double value, representing the regular profit margin, rounded to two decimal places, in the format: "Regular Margin: X. XX%", where X.XX denotes the calculated regular margin.

The second line prints a double value, representing the seasonal profit margin, rounded to two decimal places, in the format: "Seasonal Margin: X. XX%", where X.XX denotes the calculated seasonal margin.

The third line prints a string, indicating whether the business is profitable or not profitable, based on the regular margin.

If the regular margin is less than 10, print "Business is not profitable.". If it is 10 or greater, print "Business is profitable."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1000.0
800.0

Output: Regular Margin: 20.00%
Seasonal Margin: 30.00%
Business is profitable.

Answer

```
import java.util.Scanner;

class BusinessUtility {
    public double calculateMargin(double revenue, double cost) {
        return ((revenue - cost) / revenue) * 100;
    }
}

class SeasonalBusinessUtility extends BusinessUtility {
```

```

@Override
public double calculateMargin(double revenue, double cost) {
    double baseMargin = super.calculateMargin(revenue, cost);
    return baseMargin + 10;
}
}
class ProfitabilityChecker {
    public void checkProfitability(double regularMargin) {
        if (regularMargin >= 10)
            System.out.println("Business is profitable.");
        else
            System.out.println("Business is not profitable.");
    }
}
class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double revenue = scanner.nextDouble();
        double cost = scanner.nextDouble();
        BusinessUtility business = new BusinessUtility();
        SeasonalBusinessUtility seasonalBusiness = new
SeasonalBusinessUtility();
        double regularMargin = business.calculateMargin(revenue, cost);
        double seasonalMargin = seasonalBusiness.calculateMargin(revenue,
cost);

        System.out.printf("Regular Margin: %.2f%%\n", regularMargin);
        System.out.printf("Seasonal Margin: %.2f%%\n", seasonalMargin);

        ProfitabilityChecker checker = new ProfitabilityChecker();
        checker.checkProfitability(regularMargin);
        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Arun wants to calculate the age gap between the grandfather and the son

and determine the father's age after 5 years.

Your task is to assist him in developing a program using three classes: GrandFather, Father, and Son, where the GrandFather stores the grandfather's age, the Father extends GrandFather to include the father's age and calculates his age after 5 years, and Son extends Father to include the son's age and calculate the age difference between the grandfather and the son.

Input Format

The input consists of three integers representing the ages of the grandfather, father, and son, one per line.

Output Format

The first line of output prints "Grandfather and son's age gap:" followed by an integer representing the age gap between the grandfather and the son, ending with "years".

The second line prints "Father's Age:" followed by an integer representing the father's age after 5 years, ending with "years".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50

30

3

Output: Grandfather and son's age gap: 47 years

Father's Age: 35 years

Answer

```
import java.util.Scanner;
```

```
class GrandFather {  
    protected int grandfatherAge;  
    public void setGrandfatherAge(int age) {  
        this.grandfatherAge = age;  
    }  
}
```

```
    public int getGrandfatherAge() {
        return grandfatherAge;
    }
}

class Father extends GrandFather {
    protected int fatherAge;
    public void setFatherAge(int age) {
        this.fatherAge = age;
    }
    public int getFatherAge() {
        return fatherAge;
    }
    public int calculateFatherAgeAfter5Years() {
        return fatherAge + 5;
    }
}

class Son extends Father {
    private int sonAge;

    public void setSonAge(int age) {
        this.sonAge = age;
    }
    public int getSonAge() {
        return sonAge;
    }
    public int calculateGrandfatherSonAgeDifference() {
        return grandfatherAge - sonAge;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Son son = new Son();

        int grandfatherAge = scanner.nextInt();
        son.setGrandfatherAge(grandfatherAge);

        int fatherAge = scanner.nextInt();
        son.setFatherAge(fatherAge);

        int sonAge = scanner.nextInt();
        son.setSonAge(sonAge);
    }
}
```

```
System.out.println("Grandfather and son's age gap: "+  
son.calculateGrandfatherSonAgeDifference() + " years");  
  
int fatherAgeAfter5Years = son.calculateFatherAgeAfter5Years();  
System.out.println("Father's Age: " + fatherAgeAfter5Years + " years");  
}  
}
```

Status : Correct

Marks : 10/10